Parag Javale

VIBE CODING - THE FULL PROCESS

A Step-by-Step Guide for CS Students | Using TechDesk CRM as a Live Example

What this guide covers

How to go from a raw idea to a fully structured, AI-built project - without losing control.
Every phase is shown using TechDesk CRM: a real repair shop management system built with React and Supabase.
Every AI prompt used at each phase is included word-for-word. Copy them, adapt them, use them.

What You Will Learn	The 5-Phase Process
Phase 1	Raw idea → PRD document
Phase 2	PRD → Technical Specifications
Phase 3	Specs → Module-by-module documents
Phase 4	Module docs → Build with AI, step by step
Phase 5	Review, test, commit - and iterate

Who this is for: Intermediate CS students who know the basics, variables, functions, components, and want to build a real project the right way.

01 The Mindset Before You Start

You are the PM. AI is your developer. Structure is your advantage.

Why Most Student Projects Fail

It is not lack of skill. It is lack of structure. Students open ChatGPT or Claude, type 'make me a login page', and start building. Three weeks later:

- They cannot explain what half the code does.
- Features conflict with each other because there was no plan.
- A new AI session has no context, so it gives completely different code.
- There is no rollback point when something breaks.

The Root Cause

Vibe coding is not the problem. Building without documents is.
Documents are not bureaucracy, they are the instructions you give the AI. The more precise your instructions, the better the output.

Parag Javale

The 5-Phase Flow - Overview

Every serious project - student, startup, or enterprise - follows this flow. The phases do not change. Only the scale does.

01 **Raw Idea → PRD**
Write your product requirements before touching any code.

02 **PRD → Tech Specs**
Turn requirements into a technical blueprint the AI can follow.

03 **Specs → Module Docs**
Break the system into modules. Document each one before building it.

04 **Module Docs → Build**
Use focused, context-rich prompts to build each module with AI.

05 **Review → Test → Commit**
Test every output. Commit working code. Log progress. Repeat.

The TechDesk Example

TechDesk is a CRM (Customer Relationship Management) system for a small computer and printer repair shop.
It manages repair tickets, assigns engineers, tracks payments, logs activity, and reports business stats.
Built with: React (Vite) + Supabase + Recharts. Used by a real shop owner.
We will follow TechDesk through every phase in this guide.

02 Phase 1 - From Raw Idea to PRD

Turn a rough description into a structured product requirements document.

What Is a PRD?

PRD stands for Product Requirements Document. It answers three questions:

- What does this product do?
- Who is it for?
- What is in scope for this version - and what is NOT?

You write this before your first line of code. You also paste it at the start of every AI session so the AI understands the full picture.

Parag Javale

Step 1A - Write Your Raw Requirement

Start by writing down everything you know about the project in plain language. No structure needed. Just dump your thoughts. This is called a Raw Requirement.

TECHDESK RAW REQUIREMENT - WHAT THE SHOP OWNER TOLD US:

Raw Requirement (TechDesk)

I run a small computer and printer repair shop. I have 3-4 engineers working for me.
Right now, I track everything in a notebook and WhatsApp groups. It is a mess.
I want a system where I can log every repair job that comes in.
Each job should have the customer's name, phone number, what device it is, what the problem is.
I assign it to one of my engineers. The job goes through stages: Received, In Progress, Waiting for Parts, Completed, Delivered.
I want to track payment - how much the repair costs, how much the customer has paid, and what is still pending.
My engineers should be able to update the status of jobs they are working on.
I want to see a dashboard - how many jobs today, how much revenue this month, which engineer has the most jobs.
Later I might want WhatsApp notifications when a job is ready. But not right now.

Step 1B - Use AI to Convert Raw Requirement to PRD

Now take that raw requirement and give it to the AI with this prompt:

PROMPT - Convert Raw Requirement to PRD

You are a Product Manager helping a student learn how to build software the right way.

Here is a raw requirement from a client for a repair shop management system:

--- RAW REQUIREMENT ---
[Paste your raw requirement here]
--- END ---

Convert this into a structured PRD (Product Requirements Document) with these sections:
1. Product Overview (2-3 sentences)
2. Target Users
3. Core Features for Version 1 (must-have)
4. Out of Scope for Version 1 (not building yet)
5. User Roles and Permissions
6. Key Business Rules
7. Success Criteria (how do we know this works?)

Be specific. Use bullet points. Keep it to 1-2 pages.

Step 1C - The PRD Output (TechDesk Example)

Parag Javale

After running the prompt above, you get a structured PRD. Review it carefully. Add anything missing. Remove anything wrong. This is your contract with yourself.

PRD.MD - TECHDESK CRM V1

```
# PRD - TechDesk CRM v1

## 1. Product Overview
TechDesk is a repair shop management system for small electronics
repair businesses. It replaces paper notebooks and WhatsApp groups
with a structured digital workflow for tickets, engineers, and payments.

## 2. Target Users
- Shop Owner: Full access. Views reports and manages everything.
- Engineer: Views and updates assigned tickets only.

## 3. Core Features - Version 1
- Create and manage repair tickets
- Assign tickets to engineers
- Ticket status: Received / In Progress / Waiting for Parts / Done / Delivered
- Track payment per ticket (total, paid, pending)
- Activity log per ticket (who changed what, when)
- Dashboard: jobs today, revenue this month, engineer workload

## 4. Out of Scope - Version 1
- WhatsApp notifications
- Mobile app
- Inventory management
- Multiple branches

## 5. User Roles
- Owner: Create/edit/delete tickets, manage engineers, view all reports
- Engineer: View assigned tickets, update status and remarks only

## 6. Key Business Rules
- A ticket must have a customer name, device type, and problem description
- Only the owner can delete a ticket
- Payment can be partial (e.g., 50% advance, rest on delivery)

## 7. Success Criteria
- Owner can log a new ticket in under 2 minutes
- Engineer can update ticket status without owner involvement
- Owner can see revenue for the current month on the dashboard
```

Student Action

Save this as PRD.md in a /docs folder in your project.

Read it once more. If anything is missing or wrong, fix it now. It is much harder to fix after you start building.

The Out of Scope section is just as important as the features list - it stops you from building things you do not need yet.

03 Phase 2 - PRD to Technical Specifications

Turn product requirements into a technical blueprint the AI can build from.

What Are Technical Specifications?

The PRD describes WHAT to build. The Technical Specifications document describes HOW to build it:

- What tech stack (frontend, backend, database)?
- What is the database schema (tables, columns, relationships)?
- What are the main components or modules?
- How does authentication and role-based access work?
- What are the API endpoints or data access patterns?

This document becomes the bible for your AI. You paste relevant sections into every prompt.

Step 2A - Use AI to Generate Tech Specs from PRD

PROMPT - Generate Technical Specifications from PRD

You are a senior software architect helping a student build their first real project.

Here is the PRD for TechDesk CRM:
--- PRD ---
[Paste your full PRD here]
--- END PRD ---

Generate a Technical Specifications document with these sections:

1. Tech Stack - recommend what to use and why (student-friendly, no over-engineering)
2. Folder Structure - how to organise the project files
3. Database Schema - all tables with column names, types, and relationships
4. User Roles and Auth - how roles are enforced
5. Module List - what are the main modules/pages?
6. Key Data Flows - describe the 3 most important flows (e.g., creating a ticket)

Use clear headings. Be specific with table and column names.
Assume the student will use React (Vite) + Supabase.

Step 2B - Tech Specs Output (TechDesk Example)

ARCHITECTURE.MD - TECHDESK CRM

Parag Javale

```
# ARCHITECTURE.md - TechDesk CRM

## 1. Tech Stack
- Frontend:  React 18 + Vite
- Database:  Supabase (PostgreSQL + Auth + Row Level Security)
- Styling:   Tailwind CSS
- Charts:    Recharts
- Routing:   React Router v6

## 2. Folder Structure
src/
  components/     Reusable UI components
  pages/          One file per main page
  hooks/          Custom React hooks (useTickets, useAuth)
  lib/            Supabase client setup
  utils/          Helper functions
docs/
  PRD.md, ARCHITECTURE.md, PROGRESS.md, module docs

## 3. Database Schema

tickets
  id            uuid  PK
  ticket_number text  unique, auto-generated
  customer_name text  NOT NULL
  customer_phone text
  device_type   text  (Laptop / Desktop / Printer / Phone / Other)
  problem_desc  text  NOT NULL
  status        text  (received/in_progress/waiting/completed/delivered)
  assigned_to   uuid  FK → engineers.id
  total_amount  numeric(10,2)
  paid_amount   numeric(10,2)  default 0
  created_at    timestamptz    default now()
  updated_at    timestamptz    default now()
  tenant_id     uuid  FK → tenants.id   // for future SaaS

engineers
  id            uuid  PK
  name          text  NOT NULL
  email         text  unique
  phone         text
  is_active     boolean  default true
  tenant_id     uuid

ticket_activity
  id            uuid  PK
  ticket_id     uuid  FK → tickets.id
  action        text  (status_change / remark / payment_update)
  note          text
  created_by    uuid  FK → engineers.id
  created_at    timestamptz  default now()

payments
```

```
  id            uuid  PK
  ticket_id     uuid  FK → tickets.id
  amount        numeric(10,2)
  payment_mode  text  (cash / UPI / card)
  paid_at       timestamptz  default now()

## 4. Auth and Roles
- Supabase Auth handles login
- user_role stored in profiles table (owner / engineer)
- Row Level Security (RLS) on all tables
- Engineers can only SELECT/UPDATE their own tickets
- Owners have full access

## 5. Modules
- Auth (Login / Logout)
- Dashboard (stats, charts)
- Tickets (list, create, detail, edit)
- Engineers (list, add, deactivate)
- Payments (log payment, view history per ticket)
- Activity Log (per ticket timeline)
```

Why tenant_id is on every table

Even though TechDesk v1 is for one shop, tenant_id was added on day one.
When the app grows into a SaaS product (multiple shops, subscription billing), the database schema does not need to change.
This one early decision - documented in ARCHITECTURE.md - saved weeks of rework later.
Always think two steps ahead. Document it. Build one step at a time.

04 Phase 3 - Specs to Module Documents

Break your system into modules. Document each one before building it.

Why Module Documents?

Your architecture doc lists the modules. Now you need a focused document for each module that describes exactly what it does, what it looks like, and what the AI needs to build it.

Think of a Module Document as a mini-PRD for one feature. Every module doc you write before building saves you 3-4 confused AI sessions later.

TechDesk Module List

These are the 6 modules for TechDesk v1:

Parag Javale

Auth Module
- Email/password login
- Role detection (owner vs engineer)
- Protected routes
- Logout

Dashboard Module
- Today's ticket count
- Monthly revenue
- Engineer workload chart
- Recent tickets table

Tickets Module
- Ticket list with filters
- Create ticket form
- Ticket detail view
- Status update
- Assign engineer

Engineers Module
- Engineer list
- Add new engineer
- Deactivate engineer
- Tickets per engineer stat

Payments Module
- Log a payment against a ticket
- Payment mode (cash/UPI/card)
- View payment history
- Pending amount calculation

Activity Log Module
- Timeline per ticket
- Who changed what and when
- Status change history
- Remarks from engineers

Step 3A - Use AI to Generate a Module Document

For each module, use this prompt. Here we show it for the Tickets module - the most important one.

PROMPT - Generate Module Document

You are helping me build TechDesk CRM - a repair shop management system.

Here is the PRD: [paste PRD]
Here is the Architecture: [paste ARCHITECTURE.md]

Generate a detailed module document for the TICKETS MODULE with these sections:

1. Module Purpose (1 paragraph)
2. Pages / Screens in this module
3. For each screen: what data is shown, what actions the user can take
4. Component List (what React components are needed)
5. Supabase queries needed (list the main data operations)
6. Validation rules
7. Edge cases to handle
8. Build order (what to build first inside this module)

Be specific. Use the exact table and column names from the architecture doc.

Step 3B - Tickets Module Document (TechDesk Example)

DOCS/MODULES/TICKETS.MD

```
# Module: Tickets

## Purpose
Core module. Every workflow starts here. Manages the full
lifecycle of a repair job from intake to delivery.

## Screens

### 1. Ticket List (/tickets)
  - Shows all tickets in a table
  - Columns: Ticket No, Customer, Device, Status, Engineer, Pending Payment
  - Filters: by status, by engineer, by date range
  - Search: by customer name or ticket number
  - Action: click row to open detail view
  - Button: 'New Ticket' (owner only)

### 2. Create Ticket (/tickets/new)
  - Form fields: customer_name*, customer_phone, device_type*,
    problem_desc*, assigned_to, total_amount
  - Auto-generate ticket number (TDK-0001, TDK-0002...)
  - Default status: 'received'
  - On submit: insert into tickets + insert activity log entry

### 3. Ticket Detail (/tickets/:id)
  - Header: ticket number, status badge, customer info
  - Status update dropdown (engineer can update their tickets)
  - Payment summary: total / paid / pending
  - Add Payment button (opens modal)
  - Activity timeline (all changes in order)
  - Add Remark text area

## Components
  TicketTable, TicketRow, TicketFilters, TicketStatusBadge,
  CreateTicketForm, TicketDetail, PaymentSummary,
  ActivityTimeline, AddRemarkForm, PaymentModal

## Supabase Queries
  - getTickets(filters)     SELECT with joins to engineers
  - getTicketById(id)       SELECT single with payments + activity
  - createTicket(data)      INSERT + insert activity log
  - updateTicketStatus(id)  UPDATE status + insert activity log
  - assignEngineer(id, eng) UPDATE assigned_to

## Validation Rules
  - customer_name: required, min 2 chars
  - device_type: must be from enum list
  - problem_desc: required, min 10 chars
  - total_amount: numeric, >= 0

## Edge Cases
```

```
  - What if engineer is deactivated but has open tickets?
  - What if paid_amount > total_amount? (block this)
  - What if ticket number generation fails? (fallback to timestamp)

## Build Order
  1. Ticket list page (read-only first)
  2. Create ticket form
  3. Ticket detail view (display only)
  4. Status update
  5. Payment modal
  6. Activity timeline
  7. Filters and search
```

Do this for every module

Before you build any module, write its document first.
Save each one as: docs/modules/AUTH.md, docs/modules/DASHBOARD.md, etc.
These docs are what you paste into AI when building that module - not the full architecture doc.
Giving the AI too much context at once confuses it. Give it only what is relevant to the current task.

05 Phase 4 - Build Each Module with AI

Focused, context-rich prompts. One task at a time. Test before moving on.

The Golden Rule of Building

One task per prompt. Test before the next prompt.

Never ask AI to build two features in one prompt.
Never move to the next task before the current one works.
This sounds slow. It is actually faster - because you never have to debug 5 things at once.

Step 4A - Project Setup Prompt

Before any module, set up the project correctly. Do this once at the very beginning.

PROMPT - Project Setup

Create a new React + Vite project for TechDesk CRM.

Setup requirements:
- React 18 + Vite
- Tailwind CSS configured
- React Router v6 installed
- Supabase JS client installed

Parag Javale

- Create src/lib/supabase.js with client setup (I will add my keys separately)
- Create this folder structure:
  src/components/ src/pages/ src/hooks/ src/lib/ src/utils/
- Create a basic App.jsx with router setup and placeholder routes for:
  /login, /dashboard, /tickets, /engineers

Show me every file to create and its full content.

Step 4B - Supabase Schema Setup Prompt

Before building any UI, create your database tables.

PROMPT - Create Supabase Schema

I am using Supabase for TechDesk CRM. Generate the SQL to create these tables.
Run this in the Supabase SQL editor.

Tables needed (from my ARCHITECTURE.md):
[Paste the database schema section from your architecture doc]

Also include:
- Enable Row Level Security on all tables
- Basic RLS policy: authenticated users can read tickets
- Trigger to auto-update updated_at on tickets table

Format as clean SQL I can copy-paste into Supabase SQL editor.

Step 4C - Building the First Module: Ticket List

With setup done, start with the most important screen. Always start with READ (display data) before CREATE/UPDATE.

PROMPT - Build Ticket List Page

I am building TechDesk CRM with React + Vite + Supabase + Tailwind.

My tickets table has these columns:
id, ticket_number, customer_name, customer_phone, device_type, problem_desc,
status, assigned_to (FK to engineers), total_amount, paid_amount, created_at

Build the Ticket List page (/tickets) with:
- Fetch all tickets from Supabase, join with engineers table to get engineer name
- Show in a table: Ticket No | Customer | Device | Status | Engineer | Pending
- Status shown as a colored badge (received=blue, in_progress=yellow,
  waiting=orange, completed=green, delivered=gray)

Parag Javale

- Pending = total_amount - paid_amount
- A 'New Ticket' button in the top right (just link to /tickets/new for now)
- Loading state while fetching
- Empty state if no tickets

Create: src/pages/TicketsPage.jsx and src/components/TicketStatusBadge.jsx
Use Tailwind for styling. Keep it clean and readable.

Step 4D - Building the Create Ticket Form

Once the list is working and tested, build the form to add new tickets.

PROMPT - Build Create Ticket Form

I have a working Ticket List page in my TechDesk CRM.

Now build the Create Ticket form (/tickets/new) with:

Fields:
- customer_name (text, required)
- customer_phone (text, optional)
- device_type (select: Laptop / Desktop / Printer / Phone / Other, required)
- problem_desc (textarea, required, min 10 chars)
- assigned_to (select dropdown fetched from engineers table, optional)
- total_amount (number, optional, default 0)

On submit:
1. Validate all required fields
2. Auto-generate ticket_number as 'TDK-' + padded count (e.g., TDK-0001)
3. Set status to 'received' by default
4. Insert into tickets table
5. Insert an activity log entry: 'Ticket created'
6. Redirect to /tickets on success
7. Show error message if insert fails

Create: src/pages/CreateTicketPage.jsx
Show the full file. Use Tailwind. Do not change any existing files.

Step 4E - Debugging Prompt

When something does not work, use this structure. Never just say 'it is broken'.

PROMPT - Bug Fix (structured)

I am building TechDesk CRM. The Create Ticket form is not submitting correctly.

Here is the relevant code:
[paste the function or component that has the bug]

What is happening:
- I fill in all fields and click Submit
- The form freezes, no error shows, nothing is inserted in Supabase

What I expect to happen:
- Data should be inserted into the tickets table
- User should be redirected to /tickets

My Supabase insert code is:
[paste your insert code]

Please: find the bug, fix it, and explain what was wrong in 2 sentences.

Step 4F - Adding a Feature to Existing Code

PROMPT - Add Feature to Existing Component

I have a working Ticket List page in TechDesk CRM.
Here is the current code: [paste TicketsPage.jsx]

Add a search bar at the top that:
- Filters tickets by customer name OR ticket number
- Filters in real time as the user types (client-side, no new Supabase query)
- Shows a 'No results found' message if nothing matches

Rules:
- Do NOT change the table columns or styling
- Do NOT add any new dependencies
- Add the search state and filter logic inside the existing component

Show me only the changed/added lines, not the full file.

06 Phase 5 - Review, Test, Commit, Log

The loop that keeps your project alive and your sanity intact.

The End-of-Session Loop

After every coding session - before you close your laptop - do this in order:

Parag Javale

01 **Test what was built**
Run the app. Click through every change. Try to break it - empty inputs, wrong data types, fast clicking.

02 **Commit with a real message**
git add . and git commit -m 'feat: add ticket list page with status badges'

03 **Push to GitHub**
git push. Your laptop is not a server. Never end a session without pushing.

04 **Update PROGRESS.md**
5 minutes. What did you finish? What broke and how did you fix it? What is next?

PROGRESS.md Template

DOCS/PROGRESS.MD - FILL THIS EVERY SESSION

```
## 2025-06-04 (Wednesday)

### Done
- Built Ticket List page (/tickets)
- Added TicketStatusBadge component with color coding
- Supabase query working, joins engineers table correctly

### Broke / Fixed
- Supabase join was returning null for engineer name
  Fixed: changed .select('*, engineers(name)') to correct foreign key format

### Blocked
- Ticket number auto-generation logic not figured out yet
  Plan: query MAX(ticket_number) and increment

### Committed
- feat: add ticket list page with status badges and engineer join

### Tomorrow
- [ ] Create Ticket form (/tickets/new)
- [ ] Auto ticket number generation
- [ ] Form validation
```

Git Commit Message Guide

Parag Javale

Type	When to use it
feat:	Adding a new feature or page
fix:	Fixing a bug
docs:	Updating any .md documentation file
refactor:	Reorganising code without changing behaviour
style:	CSS/Tailwind styling changes only
chore:	Removing console.logs, cleanup, formatting

Commit BEFORE every big AI change

Before asking AI to rewrite a component or change a lot of code:
git add . && git commit -m 'chore: safe checkpoint before refactor'
This gives you a rollback point if the AI breaks something.
This single habit will save you hours of pain.

07 Prompt Library - Copy and Adapt

All prompts in one place. Replace [brackets] with your own content.

How to Use These Prompts

- Replace every [bracket] with your project's actual content.
- Always include your PRD or module doc as context.
- Specify what NOT to change - this prevents AI from rewriting things you did not ask for.
- Ask for explanations when you do not understand the output.

Prompt 1 - Start a New AI Session

PROMPT - Context Reset (use at start of every session)

I am building [PROJECT NAME]. Here is a summary of what it is:
[2-3 sentence description from your PRD]

Tech stack: [your stack]

What I have built so far:
[Paste last 3-4 lines from your PROGRESS.md]

Today I want to work on: [specific task]

Before we start, confirm you understand the project and the task.

Parag Javale

Prompt 2 - Explain AI Code You Do Not Understand

PROMPT - Explain Code

Explain this code to me like I am an intermediate developer:
[paste code]

I want to understand:
1. What does this do, step by step?
2. Why is it written this way?
3. Are there any patterns here I should know the name of?

Keep it simple. Use an analogy if it helps.

Prompt 3 - Code Review

PROMPT - Code Review

Review this component I wrote for [PROJECT NAME]:
[paste code]

Check for:
1. Any bugs or edge cases I missed
2. Anything that would break under unexpected input
3. One improvement suggestion (not a rewrite, just one thing)

Do not rewrite it. Just tell me what to fix.

Prompt 4 - Generate a Supabase Query

PROMPT - Supabase Query

I am using Supabase with React.

My table structure: [paste relevant table schema]

I need a query that:
[describe exactly what data you need]

Return it as a React custom hook (useXxx) that:
- Fetches data on mount
- Returns { data, loading, error }
- Handles the error case cleanly

Prompt 5 - Build a Full Module from Module Doc

Parag Javale

PROMPT - Build Module from Spec

I am building TechDesk CRM with React + Vite + Supabase + Tailwind.

Here is the module document for [MODULE NAME]:
[paste the module doc]

Build this module following the build order in the doc.
Start with task 1 only: [first item in build order]

Rules:
- Create new files only. Do not edit existing files.
- Use Tailwind only (no inline styles, no CSS files)
- Match the exact field names from the module doc
- Show full file content for every file you create

Prompt 6 - Final Checklist Before Demo

PROMPT - Pre-Demo Review

I am about to demo TechDesk CRM. Review this component for production readiness:
[paste the most important component]

Check:
1. Any console.log statements to remove?
2. Any missing loading or error states?
3. Does it handle empty data gracefully?
4. Any obvious security issues (e.g., client-side role checks that should be server-side)?

Give me a short checklist of what to fix.

08 Quick Reference - Keep This Open While Building

Checklists, rules, and the document stack at a glance.

Document Stack - Create These First

File	What it contains
README.md	What the project does, how to run it locally, tech stack, your name
docs/PRD.md	Product requirements - what to build, who for, what is out of scope
docs/ARCHITECTURE.md	Tech stack, folder structure, database schema, module list

Parag Javale

File	What it contains
docs/PROGRESS.md	Daily log - what was done, what broke, what is next
docs/modules/TICKETS.md	Detailed spec for the Tickets module
docs/modules/DASHBOARD.md	Detailed spec for the Dashboard module
docs/PROMPTS.md	Good prompts that worked - paste them here for reuse
docs/DECISIONS.md	Why you chose X over Y - write this as you make choices

Session Start Checklist

- git pull (get latest from GitHub)
- Read PROGRESS.md from last session
- Paste PRD + today's goal into first AI message
- git commit current state before any big AI change

Session End Checklist

- Test everything you built (run the app, click every change)
- git add . && git commit -m 'meaningful message'
- git push
- Update PROGRESS.md (5 minutes - done / broke / next)

Before Demo / Submission

- README.md is accurate and has working setup instructions
- App runs from a clean git clone (test this on a different folder)
- No console.log statements in the code
- Edge cases tested: empty state, long text, zero results
- You can explain every file and every component

The 7 Rules - Read Every Day

Rule	Why it matters
01 - One task per prompt	Focused prompts give reliable output. Vague prompts give random output.
02 - Understand before accepting	If you cannot explain the code, you do not own it. Ask AI to explain.
03 - Document as you go	5 minutes now saves 2 hours later. Update PROGRESS.md every session.
04 - Freeze scope at 60%	A polished core beats a feature-dumped mess. Stop adding, start refining.
05 - Test edge cases	Empty inputs, wrong types, no results. AI never thinks about these.

Parag Javale

Rule	Why it matters
06 - Fresh context every session	Paste your PRD at the start of every new AI chat. Always.
07 - You own the code	Be ready to explain every file in an interview. If you cannot, learn it now.

The builders who win are not the ones who prompt the most.

They are the ones who think the clearest - and have the docs to prove it.